UNIVERSIDAD DE SANTANDER

Bases de Datos II

Taller 1 — Empresa "Huellitas"

Robert Caicedo Sanchez

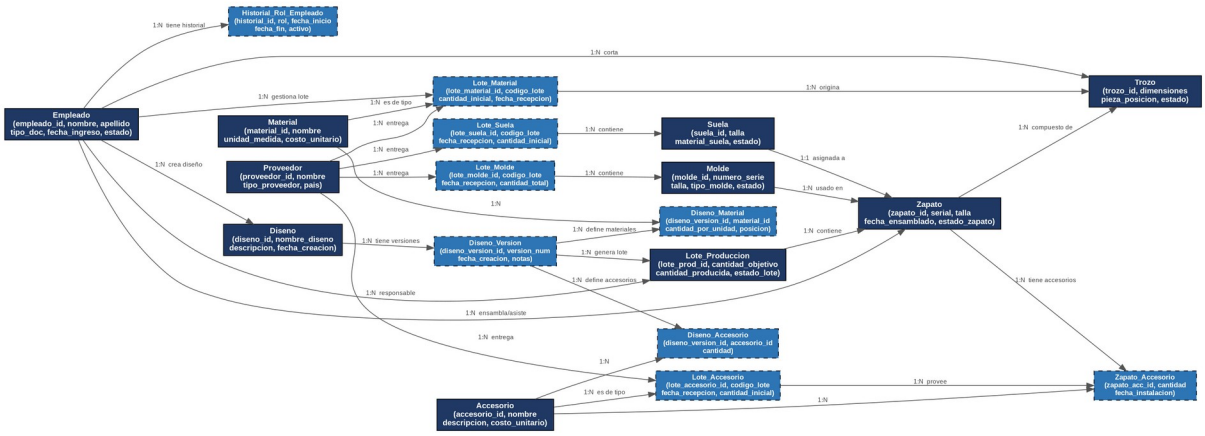
Kaleth Ceballos

Bucaramanga, 2026

Actividad 1: Modelo Entidad-Relación

A continuación se presenta el modelo de datos propuesto para el sistema de la empresa Huellitas, incluyendo las entidades, atributos y relaciones identificadas a partir del caso de estudio.

1.1 Diagrama Entidad-Relación



En el diagrama: los rectángulos de color azul oscuro representan entidades fuertes y los de azul claro representan entidades débiles o tablas asociativas. Las flechas indican las relaciones con su cardinalidad (1:N o 1:1).

1.2 Entidades y Atributos

Empleado		
Campo	**Tipo**	**Descripción**
empleado_id (PK)	INT IDENTITY	Identificador único del empleado
tipo_documento	NVARCHAR(20)	CC, CE, pasaporte, etc.
numero_documento	NVARCHAR(50)	Número del documento
nombre	NVARCHAR(100)	Nombre del empleado

apellido	NVARCHAR(100)	Apellido del empleado
fecha_nacimiento	DATE	Fecha de nacimiento
telefono	NVARCHAR(50)	Teléfono de contacto
email	NVARCHAR(150)	Correo electrónico
fecha_ingreso	DATE	Fecha de ingreso a la empresa
estado_empleado	NVARCHAR(50)	activo / suspendido / retirado

Historial_Rol_Empleado

Campo	Tipo	Descripción
historial_id (PK)	INT IDENTITY	Identificador del registro
empleado_id (FK)	INT NOT NULL	Referencia a Empleado
rol	NVARCHAR(50)	maestro / ayudante / cortador
fecha_inicio	DATETIME2	Inicio del periodo con ese rol
fecha_fin	DATETIME2 NULL	Fin del periodo (NULL = activo)
cambiado_por (FK)	INT NULL	Empleado que registró el cambio
motivo	NVARCHAR(300)	Razón del cambio de rol
activo	BIT	1 si el rol es el vigente

Proveedor

Campo	Tipo	Descripción
proveedor_id (PK)	INT IDENTITY	Identificador del proveedor
nombre_proveedor	NVARCHAR(150)	Razón social
tipo_proveedor	NVARCHAR(50)	materia_prima / suelas / moldes / accesorios
contacto_nombre	NVARCHAR(100)	Nombre del contacto
contacto_telefono	NVARCHAR(50)	Teléfono del contacto
pais	NVARCHAR(80)	País de origen

Material

Campo	Tipo	Descripción
material_id (PK)	INT IDENTITY	Identificador del material
nombre	NVARCHAR(150)	Nombre del material
descripcion	NVARCHAR(300)	Descripción detallada
unidad_medida	NVARCHAR(20)	m2, kg, m, etc.
costo_unitario_estimado	DECIMAL(12,2)	Costo estimado por unidad

Lote_Material

Campo	Tipo	Descripción
lote_material_id (PK)	INT IDENTITY	Identificador del lote
material_id (FK)	INT NOT NULL	Referencia a Material
proveedor_id (FK)	INT NULL	Referencia a Proveedor
codigo_lote	NVARCHAR(100)	Código externo del lote
cantidad_inicial	DECIMAL(12,3)	Cantidad al recibir

cantidad_disponible	DECIMAL(12,3)	Cantidad restante
fecha_recepcion	DATE	Fecha de entrada al inventario
fecha_agotado	DATE NULL	Fecha en que se agotó (si aplica)
gestionado_por (FK)	INT NULL	Ayudante responsable

Lote_Molde

Campo	Tipo	Descripción
lote_molde_id (PK)	INT IDENTITY	Identificador del lote
proveedor_id (FK)	INT NULL	Referencia a Proveedor
codigo_lote	NVARCHAR(100)	Código del lote
fecha_recepcion	DATE	Fecha de recepción
cantidad_total	INT	Total de moldes en el lote
cantidad_disponible	INT	Moldes aún disponibles

Molde

Campo	Tipo	Descripción
molde_id (PK)	INT IDENTITY	Identificador del molde
lote_molde_id (FK)	INT NULL	Lote al que pertenece
numero_serie	NVARCHAR(100)	Número de serie (único)
talla	INT	Talla del molde
tipo_molde	NVARCHAR(100)	empeine, plantilla, etc.
material_molde	NVARCHAR(100)	Material de fabricación
uso_acumulado	INT	Veces que se ha usado
estado	NVARCHAR(50)	operativo / reparacion / retirado

Lote_Suela

Campo	Tipo	Descripción
lote_suela_id (PK)	INT IDENTITY	Identificador del lote
proveedor_id (FK)	INT NULL	Referencia a Proveedor
codigo_lote	NVARCHAR(100)	Código del lote
fecha_recepcion	DATE	Fecha de recepción
cantidad_inicial	INT	Cantidad inicial
cantidad_disponible	INT	Cantidad disponible

Suela

Campo	Tipo	Descripción
suela_id (PK)	INT IDENTITY	Identificador de la suela
lote_suela_id (FK)	INT NOT NULL	Lote de origen
talla	INT	Talla de la suela
material_suela	NVARCHAR(100)	Material de fabricación
estado	NVARCHAR(50)	nuevo / usada / defectuosa

Accesorio		
Campo	**Tipo**	**Descripción**
accesorio_id (PK)	INT IDENTITY	Identificador del accesorio
nombre	NVARCHAR(150)	Nombre del accesorio
descripcion	NVARCHAR(300)	Descripción
proveedor_id (FK)	INT NULL	Proveedor del accesorio
costo_unitario	DECIMAL(12,2)	Costo por unidad

Lote_Accesorio		
Campo	**Tipo**	**Descripción**
lote_accesorio_id (PK)	INT IDENTITY	Identificador del lote
accesorio_id (FK)	INT NULL	Tipo de accesorio
proveedor_id (FK)	INT NULL	Proveedor
codigo_lote	NVARCHAR(100)	Código del lote
fecha_recepcion	DATE	Fecha de recepción
cantidad_inicial	INT	Cantidad inicial
cantidad_disponible	INT	Cantidad disponible

Diseno		
Campo	**Tipo**	**Descripción**
diseno_id (PK)	INT IDENTITY	Identificador del diseño
nombre_diseno	NVARCHAR(150)	Nombre del diseño
descripcion	NVARCHAR(MAX)	Descripción del diseño
creado_por (FK)	INT NULL	Maestro que lo creó
fecha_creacion	DATETIME2	Fecha de creación

Diseno_Version		
Campo	**Tipo**	**Descripción**
diseno_version_id (PK)	INT IDENTITY	Identificador de la versión
diseno_id (FK)	INT NOT NULL	Diseño al que pertenece
version_num	INT NOT NULL	Número de versión (1, 2, 3…)
creado_por (FK)	INT NULL	Empleado que creó la versión
notas	NVARCHAR(MAX)	Notas del cambio

Diseno_Material (tabla asociativa)		
Campo	**Tipo**	**Descripción**
diseno_version_id (FK)	INT NOT NULL	Versión del diseño
material_id (FK)	INT NOT NULL	Material usado
cantidad_por_unidad	DECIMAL(10,3)	Cantidad por zapato
posicion	NVARCHAR(100)	Posición en el zapato (empeine, forro, etc.)

Diseno_Accesorio (tabla asociativa)		
Campo	**Tipo**	**Descripción**
diseno_version_id (FK)	INT NOT NULL	Versión del diseño
accesorio_id (FK)	INT NOT NULL	Accesorio incluido
cantidad	INT	Cantidad por zapato

Lote_Produccion		
Campo	**Tipo**	**Descripción**
lote_prod_id (PK)	INT IDENTITY	Identificador del lote
diseno_version_id (FK)	INT NOT NULL	Versión del diseño
codigo_lote_prod	NVARCHAR(100)	Código interno
fecha_inicio	DATETIME2	Inicio de producción
cantidad_objetivo	INT NOT NULL	Zapatos a fabricar
cantidad_producida	INT	Zapatos terminados
estado_lote	NVARCHAR(50)	planificado / en_produccion / terminado
responsable_maestro_id (FK)	INT NULL	Maestro responsable

Zapato		
Campo	**Tipo**	**Descripción**
zapato_id (PK)	INT IDENTITY	Identificador del zapato
serial	NVARCHAR(150)	Serial único del zapato
lote_prod_id (FK)	INT NOT NULL	Lote de producción
diseno_version_id (FK)	INT NOT NULL	Versión del diseño
talla	INT	Talla del zapato
fecha_ensamblado	DATETIME2	Fecha de ensamble
maestro_id (FK)	INT NULL	Maestro que lo fabricó
ayudante_id (FK)	INT NULL	Ayudante que asistió
molde_usado_id (FK)	INT NULL	Molde utilizado
suela_usada_id (FK)	INT NULL	Suela utilizada
estado_zapato	NVARCHAR(50)	ensamblado / rechazado / entregado

Trozo		
Campo	**Tipo**	**Descripción**
trozo_id (PK)	INT IDENTITY	Identificador del trozo
material_id (FK)	INT NOT NULL	Material del trozo
lote_material_id (FK)	INT NULL	Lote de donde proviene
cortador_id (FK)	INT NULL	Cortador que lo generó
zapato_id (FK)	INT NULL	Zapato al que fue asignado
fecha_corte	DATETIME2	Fecha en que se cortó
dimensiones	NVARCHAR(100)	Dimensiones del trozo
pieza_posicion	NVARCHAR(100)	Posición en el zapato
estado	NVARCHAR(50)	bueno / defectuoso

Zapato_Accesorio		
Campo	**Tipo**	**Descripción**
zapato_accesorio_id (PK)	INT IDENTITY	Identificador del registro
zapato_id (FK)	INT NOT NULL	Zapato al que se instaló
accesorio_id (FK)	INT NOT NULL	Accesorio instalado
lote_accesorio_id (FK)	INT NULL	Lote del accesorio
cantidad	INT	Cantidad instalada
instalado_por (FK)	INT NULL	Empleado que instaló
fecha_instalacion	DATETIME2	Fecha de instalación

1.3 Relaciones y Cardinalidades

- Empleado 1:N Historial_Rol_Empleado — un empleado puede tener varios roles a lo largo del tiempo.
- Proveedor 1:N Lote_Material, Lote_Molde, Lote_Suela, Lote_Accesorio — un proveedor puede entregar múltiples lotes.
- Lote_Material 1:N Trozo — de un lote se generan muchos trozos.
- Molde 1:N Zapato — un mismo molde puede usarse para varios zapatos.
- Diseno 1:N Diseno_Version — cada diseño puede tener múltiples versiones.
- Diseno_Version 1:N Lote_Produccion — una versión puede originar varios lotes.
- Lote_Produccion 1:N Zapato — un lote contiene muchos zapatos.
- Zapato N:M Material — a través de Trozo (un zapato usa varios trozos de distintos materiales).
- Zapato N:M Accesorio — a través de Zapato_Accesorio.

1.4 Normalización (1FN a 6FN)

Primera Forma Normal (1FN)

Todos los atributos son atómicos y no hay grupos repetidos en una sola columna. Las colecciones como materiales de un diseño, accesorios de un zapato y trozos se modelaron en tablas asociativas independientes: Diseno_Material, Diseno_Accesorio y Zapato_Accesorio. Esto garantiza que cada celda contiene un único valor.

Segunda Forma Normal (2FN)

Se cumple porque en las tablas con clave compuesta todos los atributos dependen de la clave completa. Por ejemplo, en Diseno_Material la PK es (diseno_version_id, material_id) y el campo cantidad_por_unidad depende de ambas columnas juntas, no solo de una. Datos propios de Material como unidad_medida permanecen en su propia tabla.

Tercera Forma Normal (3FN)

No existen dependencias transitivas. Los datos del proveedor están en la tabla Proveedor y los lotes solo almacenan el proveedor_id como FK, sin repetir campos como nombre_proveedor. De forma similar, la información del empleado no se duplica en tablas de zapatos o trozos.

Forma Normal de Boyce-Codd (BCNF)

Todo determinante es una clave candidata. En Molde, el campo numero_serie es único (clave candidata), por lo que no se producen dependencias sobre determinantes no clave. El modelo supera este requisito para las tablas revisadas.

Cuarta Forma Normal (4FN)

Se evitaron las dependencias multivaloradas. Al separar los materiales y accesorios de un diseño en Diseno_Material y Diseno_Accesorio respectivamente, se previene la generación de combinaciones cartesianas innecesarias que ocurrirían si ambas colecciones estuvieran en una sola tabla.

Quinta Forma Normal (5FN)

Las tablas asociativas (Zapato_Accesorio, Zapato_Trozo) representan relaciones directas y no hay dependencias de unión ocultas. Toda la información puede reconstruirse mediante joins naturales sin pérdida de datos.

Sexta Forma Normal (6FN)

La 6FN aplica cuando se necesita modelar la variación temporal de atributos individuales. En este diseño se implementó de manera parcial mediante Historial_Rol_Empleado, que registra el rol de un empleado con sus fechas de inicio y fin. Para otros atributos cambiantes como la cantidad disponible de un lote, se podría crear una tabla de historial adicional si se requiere auditoría a nivel de dato individual.

Actividad 2: Análisis del Modelo

A) Cambio de rol: de ayudante a maestro zapatero

Cuando un empleado cambia de rol, los registros históricos no se ven afectados porque el empleado_id sigue siendo el mismo. Los zapatos que ya se fabricaron, los trozos que el empleado cortó y los lotes en los que participó siguen apuntando al mismo empleado_id. Lo único que cambia es el rol vigente.

El diseño utiliza la tabla Historial_Rol_Empleado para gestionar esto. Cuando se hace el cambio se cierran el registro activo (se asigna fecha_fin) y se inserta uno nuevo con el nuevo rol. De esta forma se puede consultar qué rol tenía el empleado en cualquier fecha pasada.

Script T-SQL — Promover empleado de ayudante a maestro

```
DECLARE @EmpleadoId   INT          = 5;
DECLARE @NuevoRol     NVARCHAR(50) = 'maestro';
DECLARE @CambiadoPor  INT          = 1;   -- admin que hace el cambio
DECLARE @Motivo       NVARCHAR(400)= 'Ascenso por evaluacion de desempeno';

BEGIN TRY
  BEGIN TRAN;

  -- 1. Cerrar el periodo de rol actual
  UPDATE dbo.Historial_Rol_Empleado
  SET    fecha_fin = SYSDATETIME(),
         activo    = 0
  WHERE  empleado_id = @EmpleadoId
  AND    fecha_fin IS NULL;

  -- 2. Insertar el nuevo periodo
  INSERT INTO dbo.Historial_Rol_Empleado
         (empleado_id, rol, fecha_inicio, activo, cambiado_por, motivo)
  VALUES (@EmpleadoId, @NuevoRol, SYSDATETIME(), 1, @CambiadoPor, @Motivo);

  COMMIT;
END TRY
```

```
BEGIN CATCH
  IF XACT_STATE() <> 0 ROLLBACK;
  THROW;
END CATCH;
```

Consulta: ¿qué rol tenía el maestro cuando ensambló un zapato?

```
-- Verificar qué rol tenía el empleado en la fecha de ensamble del zapato 123
SELECT h.rol, h.fecha_inicio, h.fecha_fin
FROM   dbo.Zapato z
JOIN   dbo.Historial_Rol_Empleado h ON h.empleado_id = z.maestro_id
WHERE  z.zapato_id = 123
AND    h.fecha_inicio <= ISNULL(z.fecha_ensamblado, SYSDATETIME())
AND   (h.fecha_fin IS NULL OR h.fecha_fin >= ISNULL(z.fecha_ensamblado,
SYSDATETIME()));
```

Los registros anteriores (zapatos ensamblados, trozos, lotes) permanecen intactos porque siguen referenciando el mismo empleado_id. El historial permite saber el rol exacto en cada momento sin tocar los datos existentes.

B) Cambio de diseño a mitad de producción (2 → 3 trozos)

Si a mitad de un lote de producción se decide cambiar el diseño —por ejemplo, pasar de 2 a 3 trozos con materiales distintos— no se debe modificar el diseño original porque eso afectaría la trazabilidad de los zapatos ya fabricados.

La solución es crear una nueva Diseno_Version con los cambios requeridos. Los zapatos ya producidos en el lote original mantienen su diseno_version_id anterior. Los restantes se asocian a un nuevo Lote_Produccion que apunta a la nueva versión. De esta forma ambas partes del lote quedan perfectamente trazables.

Script T-SQL — Crear nueva versión y repartir el lote

```
DECLARE @DisenoId       INT          = 10;
DECLARE @LoteOldId      INT          = 20;
DECLARE @CreadoPor      INT          = 1;
DECLARE @Nota           NVARCHAR(MAX)= 'Cambio a 3 trozos: se agrega refuerzo PU';
DECLARE @NuevoMaterialId INT         = 7;   -- material nuevo (debe existir ya)

BEGIN TRY
  BEGIN TRAN;

  -- 1. Calcular el número de la nueva versión
  DECLARE @NewVerNum INT = ISNULL(
    (SELECT MAX(version_num) FROM dbo.Diseno_Version WHERE diseno_id = @DisenoId), 0)
+ 1;

  INSERT INTO dbo.Diseno_Version (diseno_id, version_num, creado_por, notas)
  VALUES (@DisenoId, @NewVerNum, @CreadoPor, @Nota);
  DECLARE @NewVersionId INT = SCOPE_IDENTITY();

  -- 2. Copiar los materiales de la versión anterior
  DECLARE @PrevVerId INT = (SELECT TOP 1 diseno_version_id FROM dbo.Diseno_Version
```

```
    WHERE diseno_id = @DisenoId AND version_num = @NewVerNum - 1);

  IF @PrevVerId IS NOT NULL
    INSERT INTO dbo.Diseno_Material (diseno_version_id, material_id,
                                     cantidad_por_unidad, posicion)
    SELECT @NewVersionId, material_id, cantidad_por_unidad, posicion
    FROM   dbo.Diseno_Material
    WHERE  diseno_version_id = @PrevVerId;

  -- 3. Agregar el nuevo material (tercer trozo)
  INSERT INTO dbo.Diseno_Material (diseno_version_id, material_id,
                                   cantidad_por_unidad, posicion)
  VALUES (@NewVersionId, @NuevoMaterialId, 0.05, 'refuerzo');

  -- 4. Dividir el lote: calcular zapatos restantes
  DECLARE @Obj  INT, @Prod INT;
  SELECT @Obj  = cantidad_objetivo,
         @Prod = cantidad_producida
  FROM   dbo.Lote_Produccion WHERE lote_prod_id = @LoteOldId;

  DECLARE @Restantes INT = @Obj - ISNULL(@Prod, 0);

  IF @Restantes > 0
  BEGIN
    -- Cerrar el lote original en la cantidad ya producida
    UPDATE dbo.Lote_Produccion
    SET    cantidad_objetivo = ISNULL(@Prod, 0),
           estado_lote       = 'cerrado_parcial'
    WHERE  lote_prod_id = @LoteOldId;

    -- Crear el nuevo lote para la producción restante
    INSERT INTO dbo.Lote_Produccion
           (diseno_version_id, fecha_inicio, cantidad_objetivo, estado_lote)
    VALUES (@NewVersionId, SYSDATETIME(), @Restantes, 'planificado');
  END

  COMMIT;
END TRY
BEGIN CATCH
  IF XACT_STATE() <> 0 ROLLBACK;
  THROW;
END CATCH;
```

Con este enfoque los lotes que ya están terminados no se tocan. Cada zapato ya ensamblado conserva su diseno_version_id original y los nuevos zapatos del lote restante se fabrican con la versión actualizada del diseño.

Actividad 3: Creación de la Base de Datos y Operaciones

3.1 Creación del esquema

```
IF DB_ID('TallerZapateria') IS NULL
  CREATE DATABASE TallerZapateria;
GO
USE TallerZapateria;
GO

-- Tablas catálogo
CREATE TABLE Proveedor (
  proveedor_id     INT IDENTITY PRIMARY KEY,
  nombre_proveedor NVARCHAR(150) NOT NULL,
  tipo_proveedor   NVARCHAR(50)  NULL,
  contacto_nombre  NVARCHAR(100) NULL,
  contacto_telefono NVARCHAR(50) NULL,
  contacto_email   NVARCHAR(150) NULL,
  pais             NVARCHAR(80)  NULL
);
GO

CREATE TABLE Material (
  material_id              INT IDENTITY PRIMARY KEY,
  nombre                   NVARCHAR(150) NOT NULL UNIQUE,
  descripcion              NVARCHAR(300) NULL,
  unidad_medida            NVARCHAR(20)  NULL,
  costo_unitario_estimado  DECIMAL(12,2) NULL
);
GO

CREATE TABLE Accesorio (
  accesorio_id    INT IDENTITY PRIMARY KEY,
  nombre          NVARCHAR(150) NOT NULL UNIQUE,
  descripcion     NVARCHAR(300) NULL,
  proveedor_id    INT NULL REFERENCES Proveedor(proveedor_id),
  costo_unitario  DECIMAL(12,2) NULL
);
GO

-- Empleado e historial de roles
CREATE TABLE Empleado (
  empleado_id     INT IDENTITY PRIMARY KEY,
  tipo_documento  NVARCHAR(20)  NULL,
  numero_documento NVARCHAR(50) NULL,
  nombre          NVARCHAR(100) NOT NULL,
  apellido        NVARCHAR(100) NOT NULL,
  fecha_nacimiento DATE         NULL,
  telefono        NVARCHAR(50)  NULL,
  email           NVARCHAR(150) NULL,
  fecha_ingreso   DATE          NULL,
  estado_empleado NVARCHAR(50)  NULL
);
```

```
GO

CREATE TABLE Historial_Rol_Empleado (
  historial_id INT IDENTITY PRIMARY KEY,
  empleado_id  INT          NOT NULL REFERENCES Empleado(empleado_id),
  rol          NVARCHAR(50) NOT NULL,
  fecha_inicio DATETIME2    NOT NULL DEFAULT SYSUTCDATETIME(),
  fecha_fin    DATETIME2    NULL,
  cambiado_por INT          NULL     REFERENCES Empleado(empleado_id),
  motivo       NVARCHAR(300)NULL,
  activo       BIT          NOT NULL DEFAULT 1
);
GO

-- Lotes de materiales, moldes, suelas y accesorios
CREATE TABLE Lote_Material (
  lote_material_id  INT IDENTITY PRIMARY KEY,
  material_id       INT NOT NULL REFERENCES Material(material_id),
  proveedor_id      INT NULL     REFERENCES Proveedor(proveedor_id),
  codigo_lote       NVARCHAR(100) NOT NULL,
  cantidad_inicial  DECIMAL(12,3) NOT NULL,
  cantidad_disponible DECIMAL(12,3) NOT NULL,
  fecha_recepcion   DATE NULL,
  fecha_agotado     DATE NULL,
  gestionado_por    INT  NULL REFERENCES Empleado(empleado_id)
);
GO

CREATE TABLE Lote_Molde (
  lote_molde_id     INT IDENTITY PRIMARY KEY,
  proveedor_id      INT NULL REFERENCES Proveedor(proveedor_id),
  codigo_lote       NVARCHAR(100) NULL,
  fecha_recepcion   DATE NULL,
  cantidad_total    INT  NULL,
  cantidad_disponible INT NULL
);
GO

CREATE TABLE Lote_Suela (
  lote_suela_id     INT IDENTITY PRIMARY KEY,
  proveedor_id      INT NULL REFERENCES Proveedor(proveedor_id),
  codigo_lote       NVARCHAR(100) NULL,
  fecha_recepcion   DATE NULL,
  cantidad_inicial  INT  NULL,
  cantidad_disponible INT NULL
);
GO

CREATE TABLE Lote_Accesorio (
  lote_accesorio_id INT IDENTITY PRIMARY KEY,
  accesorio_id      INT NULL REFERENCES Accesorio(accesorio_id),
  proveedor_id      INT NULL REFERENCES Proveedor(proveedor_id),
```

```
  codigo_lote       NVARCHAR(100) NULL,
  fecha_recepcion   DATE NULL,
  cantidad_inicial  INT  NULL,
  cantidad_disponible INT NULL
);
GO

-- Molde y Suela (entidades físicas)
CREATE TABLE Molde (
  molde_id         INT IDENTITY PRIMARY KEY,
  lote_molde_id    INT NULL REFERENCES Lote_Molde(lote_molde_id),
  numero_serie     NVARCHAR(100) NULL,
  talla            INT NULL,
  tipo_molde       NVARCHAR(100) NULL,
  material_molde   NVARCHAR(100) NULL,
  uso_acumulado    INT NULL DEFAULT 0,
  estado           NVARCHAR(50) NULL
);
CREATE UNIQUE INDEX UQ_Molde_Serie ON Molde(numero_serie) WHERE numero_serie IS NOT
NULL;
GO

CREATE TABLE Suela (
  suela_id         INT IDENTITY PRIMARY KEY,
  lote_suela_id    INT NOT NULL REFERENCES Lote_Suela(lote_suela_id),
  talla            INT NULL,
  material_suela   NVARCHAR(100) NULL,
  estado           NVARCHAR(50)  NULL
);
GO

-- Diseños y versiones
CREATE TABLE Diseno (
  diseno_id     INT IDENTITY PRIMARY KEY,
  nombre_diseno NVARCHAR(150) NOT NULL,
  descripcion   NVARCHAR(MAX) NULL,
  creado_por    INT NULL REFERENCES Empleado(empleado_id),
  fecha_creacion DATETIME2 DEFAULT SYSUTCDATETIME()
);
CREATE UNIQUE INDEX UQ_Diseno_Nombre ON Diseno(nombre_diseno);
GO

CREATE TABLE Diseno_Version (
  diseno_version_id INT IDENTITY PRIMARY KEY,
  diseno_id         INT NOT NULL REFERENCES Diseno(diseno_id),
  version_num       INT NOT NULL,
  creado_por        INT NULL REFERENCES Empleado(empleado_id),
  fecha_creacion    DATETIME2 DEFAULT SYSUTCDATETIME(),
  notas             NVARCHAR(MAX) NULL,
  CONSTRAINT UQ_DV UNIQUE (diseno_id, version_num)
);
GO
```

```
CREATE TABLE Diseno_Material (
  diseno_version_id INT           NOT NULL REFERENCES
Diseno_Version(diseno_version_id),
  material_id       INT           NOT NULL REFERENCES Material(material_id),
  cantidad_por_unidad DECIMAL(10,3) NOT NULL,
  posicion          NVARCHAR(100) NULL,
  PRIMARY KEY (diseno_version_id, material_id)
);
GO

CREATE TABLE Diseno_Accesorio (
  diseno_version_id INT NOT NULL REFERENCES Diseno_Version(diseno_version_id),
  accesorio_id      INT NOT NULL REFERENCES Accesorio(accesorio_id),
  cantidad          INT NOT NULL DEFAULT 1,
  PRIMARY KEY (diseno_version_id, accesorio_id)
);
GO

-- Lote de producción y zapatos
CREATE TABLE Lote_Produccion (
  lote_prod_id          INT IDENTITY PRIMARY KEY,
  diseno_version_id     INT NOT NULL REFERENCES Diseno_Version(diseno_version_id),
  codigo_lote_prod      NVARCHAR(100) NULL,
  fecha_inicio          DATETIME2 NULL,
  cantidad_objetivo     INT NOT NULL,
  cantidad_producida    INT NOT NULL DEFAULT 0,
  estado_lote           NVARCHAR(50) NULL,
  responsable_maestro_id INT NULL REFERENCES Empleado(empleado_id)
);
GO

CREATE TABLE Zapato (
  zapato_id         INT IDENTITY PRIMARY KEY,
  serial            NVARCHAR(150) NOT NULL,
  lote_prod_id      INT NOT NULL REFERENCES Lote_Produccion(lote_prod_id),
  diseno_version_id INT NOT NULL REFERENCES Diseno_Version(diseno_version_id),
  talla             INT NULL,
  fecha_ensamblado  DATETIME2 NULL,
  maestro_id        INT NULL REFERENCES Empleado(empleado_id),
  ayudante_id       INT NULL REFERENCES Empleado(empleado_id),
  molde_usado_id    INT NULL REFERENCES Molde(molde_id),
  suela_usada_id    INT NULL REFERENCES Suela(suela_id),
  estado_zapato     NVARCHAR(50) NULL,
  CONSTRAINT UQ_Zapato_Serial UNIQUE (serial)
);
GO

CREATE TABLE Trozo (
  trozo_id         INT IDENTITY PRIMARY KEY,
  material_id      INT NOT NULL REFERENCES Material(material_id),
  lote_material_id INT NULL REFERENCES Lote_Material(lote_material_id),
```

```
  cortador_id      INT NULL REFERENCES Empleado(empleado_id),
  zapato_id        INT NULL REFERENCES Zapato(zapato_id),
  fecha_corte      DATETIME2 NULL,
  dimensiones      NVARCHAR(100) NULL,
  pieza_posicion   NVARCHAR(100) NULL,
  estado           NVARCHAR(50)  NULL
);
GO

CREATE TABLE Zapato_Accesorio (
  zapato_accesorio_id INT IDENTITY PRIMARY KEY,
  zapato_id           INT NOT NULL REFERENCES Zapato(zapato_id),
  accesorio_id        INT NOT NULL REFERENCES Accesorio(accesorio_id),
  lote_accesorio_id   INT NULL REFERENCES Lote_Accesorio(lote_accesorio_id),
  cantidad            INT NOT NULL DEFAULT 1,
  instalado_por       INT NULL REFERENCES Empleado(empleado_id),
  fecha_instalacion   DATETIME2 NULL
);
GO
```

3.2 Scripts de Operaciones

A) Insertar 3 maestros, 2 ayudantes y 2 cortadores

```
USE TallerZapateria;
SET NOCOUNT ON;
GO

BEGIN TRY
  BEGIN TRAN;

  -- 3 Maestros zapateros
  INSERT INTO Empleado (nombre, apellido, fecha_ingreso)
  VALUES ('Carlos',  'Gomez',    '2021-03-01'),
         ('Lucia',   'Perez',    '2020-07-15'),
         ('Juan',    'Martinez', '2019-11-10');

  -- 2 Ayudantes
  INSERT INTO Empleado (nombre, apellido, fecha_ingreso)
  VALUES ('Andres',  'Lopez',   '2023-01-05'),
         ('Mariana', 'Suarez',  '2022-06-20');

  -- 2 Cortadores
  INSERT INTO Empleado (nombre, apellido, fecha_ingreso)
  VALUES ('Pedro', 'Ramirez', '2021-08-10'),
         ('Sofia', 'Ramirez', '2022-02-25');

  -- Historial de roles
  -- Maestros: IDs 1, 2, 3
  INSERT INTO Historial_Rol_Empleado (empleado_id, rol, activo, motivo)
  VALUES (1,'maestro', 1,'Insercion inicial'),
         (2,'maestro', 1,'Insercion inicial'),
```

```
        (3,'maestro', 1,'Insercion inicial');

  -- Ayudantes: IDs 4, 5
  INSERT INTO Historial_Rol_Empleado (empleado_id, rol, activo, motivo)
  VALUES (4,'ayudante', 1,'Insercion inicial'),
         (5,'ayudante', 1,'Insercion inicial');

  -- Cortadores: IDs 6, 7
  INSERT INTO Historial_Rol_Empleado (empleado_id, rol, activo, motivo)
  VALUES (6,'cortador', 1,'Insercion inicial'),
         (7,'cortador', 1,'Insercion inicial');

  COMMIT;
END TRY
BEGIN CATCH
  ROLLBACK;
  PRINT 'Error seccion A: ' + ERROR_MESSAGE();
END CATCH;
GO
```

B) Insertar un nuevo diseño de zapato

```
BEGIN TRY
  BEGIN TRAN;

  -- Materiales necesarios
  INSERT INTO Material (nombre, descripcion, unidad_medida)
  VALUES ('Cuero natural',  'Cuero vacuno curtido', 'm2'),
         ('Tela sintetica', 'Tela poliester 600D',  'm2');

  -- Diseño principal
  INSERT INTO Diseno (nombre_diseno, descripcion, creado_por)
  VALUES ('Modelo Actividad', 'Zapato clasico para taller', 1);

  DECLARE @dis_id INT = SCOPE_IDENTITY();

  -- Version 1 del diseño
  INSERT INTO Diseno_Version (diseno_id, version_num, creado_por, notas)
  VALUES (@dis_id, 1, 1, 'Version inicial - 2 trozos');

  DECLARE @dv_id INT = SCOPE_IDENTITY();

  -- Materiales del diseño
  INSERT INTO Diseno_Material (diseno_version_id, material_id, cantidad_por_unidad,
posicion)
  VALUES (@dv_id, 1, 0.30, 'empeine'),
         (@dv_id, 2, 0.10, 'lengueta');

  COMMIT;
END TRY
BEGIN CATCH
  ROLLBACK;
```

```
  PRINT 'Error seccion B: ' + ERROR_MESSAGE();
END CATCH;
GO
```

C) Insertar lote de 10 zapatos

```
BEGIN TRY
  BEGIN TRAN;

  -- Crear un molde base
  INSERT INTO Molde (tipo_molde, material_molde, estado)
  VALUES ('plantilla', 'acero', 'operativo');
  DECLARE @molde_id INT = SCOPE_IDENTITY();

  -- Lote de produccion
  INSERT INTO Lote_Produccion
    (diseno_version_id, codigo_lote_prod, fecha_inicio, cantidad_objetivo,
     estado_lote, responsable_maestro_id)
  VALUES (1, 'LP-2026-001', SYSUTCDATETIME(), 10, 'en_produccion', 1);
  DECLARE @lote_id INT = SCOPE_IDENTITY();

  -- Insertar 10 zapatos
  DECLARE @i INT = 1;
  WHILE @i <= 10
  BEGIN
    INSERT INTO Zapato
      (serial, lote_prod_id, diseno_version_id, talla, fecha_ensamblado,
       maestro_id, estado_zapato, molde_usado_id)
    VALUES (CONCAT('ZAP-LP001-', @i), @lote_id, 1, 42,
            SYSUTCDATETIME(), 1, 'ensamblado', @molde_id);
    SET @i = @i + 1;
  END

  -- Actualizar contador del lote
  UPDATE Lote_Produccion
  SET cantidad_producida = (SELECT COUNT(*) FROM Zapato WHERE lote_prod_id = @lote_id)
  WHERE lote_prod_id = @lote_id;

  COMMIT;
END TRY
BEGIN CATCH
  ROLLBACK;
  PRINT 'Error seccion C: ' + ERROR_MESSAGE();
END CATCH;
GO
```

D) Modificar rol de ayudante a maestro zapatero

```
-- Promover a Andres Lopez (empleado_id = 4) de ayudante a maestro
BEGIN TRY
  BEGIN TRAN;
```

```
  -- Cerrar rol activo
  UPDATE Historial_Rol_Empleado
  SET    fecha_fin = SYSUTCDATETIME(), activo = 0
  WHERE  empleado_id = 4 AND activo = 1;

  -- Abrir nuevo rol
  INSERT INTO Historial_Rol_Empleado (empleado_id, rol, activo, cambiado_por, motivo)
  VALUES (4, 'maestro', 1, 1, 'Promovido de ayudante a maestro - actividad 3D');

  COMMIT;
END TRY
BEGIN CATCH
  ROLLBACK;
  PRINT 'Error seccion D: ' + ERROR_MESSAGE();
END CATCH;
GO
```

E) Actualizar diseño: agregar un accesorio nuevo

```
-- Crear el accesorio y asociarlo a una nueva version del diseño
BEGIN TRY
  BEGIN TRAN;

  INSERT INTO Accesorio (nombre, descripcion)
  VALUES ('Hebilla dorada', 'Hebilla metalica decorativa dorada');
  DECLARE @acc_id INT = SCOPE_IDENTITY();

  -- Nueva version del diseño
  INSERT INTO Diseno_Version (diseno_id, version_num, creado_por, notas)
  VALUES (1, 2, 1, 'Version 2: agrega hebilla dorada');
  DECLARE @dv2 INT = SCOPE_IDENTITY();

  -- Copiar materiales de la version anterior
  INSERT INTO Diseno_Material (diseno_version_id, material_id, cantidad_por_unidad,
posicion)
  SELECT @dv2, material_id, cantidad_por_unidad, posicion
  FROM   Diseno_Material WHERE diseno_version_id = 1;

  -- Asociar el accesorio
  INSERT INTO Diseno_Accesorio (diseno_version_id, accesorio_id, cantidad)
  VALUES (@dv2, @acc_id, 1);

  COMMIT;
END TRY
BEGIN CATCH
  ROLLBACK;
  PRINT 'Error seccion E: ' + ERROR_MESSAGE();
END CATCH;
GO
```

F) Actualizar diseño: agregar trozo de material diferente

```
BEGIN TRY
  BEGIN TRAN;

  -- Nuevo material
  INSERT INTO Material (nombre, descripcion, unidad_medida)
  VALUES ('Refuerzo PU', 'Poliuretano para refuerzo lateral', 'm2');
  DECLARE @mat_ref INT = SCOPE_IDENTITY();

  -- Nueva version del diseño (version 3)
  INSERT INTO Diseno_Version (diseno_id, version_num, creado_por, notas)
  VALUES (1, 3, 1, 'Version 3: 3 trozos, agrega refuerzo PU');
  DECLARE @dv3 INT = SCOPE_IDENTITY();

  -- Copiar materiales de la version 2
  INSERT INTO Diseno_Material (diseno_version_id, material_id, cantidad_por_unidad,
posicion)
  SELECT @dv3, material_id, cantidad_por_unidad, posicion
  FROM   Diseno_Material
  WHERE  diseno_version_id = (SELECT MAX(diseno_version_id)
                              FROM Diseno_Version WHERE diseno_id = 1
                              AND diseno_version_id < @dv3);

  -- Agregar el nuevo trozo
  INSERT INTO Diseno_Material (diseno_version_id, material_id, cantidad_por_unidad,
posicion)
  VALUES (@dv3, @mat_ref, 0.05, 'refuerzo lateral');

  COMMIT;
END TRY
BEGIN CATCH
  ROLLBACK;
  PRINT 'Error seccion F: ' + ERROR_MESSAGE();
END CATCH;
GO
```

G) Eliminar un zapato de un lote

```
-- Eliminar el quinto zapato del lote LP-2026-001
BEGIN TRY
  BEGIN TRAN;

  DECLARE @lote_act INT = (SELECT lote_prod_id FROM Lote_Produccion
                           WHERE codigo_lote_prod = 'LP-2026-001');

  DECLARE @zap_id INT = (
    SELECT zapato_id FROM (
      SELECT zapato_id, ROW_NUMBER() OVER (ORDER BY zapato_id) AS rn
      FROM   Zapato WHERE lote_prod_id = @lote_act
    ) t WHERE rn = 5
  );

  IF @zap_id IS NOT NULL
```

```
    DELETE FROM Zapato WHERE zapato_id = @zap_id;

  -- Recalcular contador
  UPDATE Lote_Produccion
  SET cantidad_producida = (SELECT COUNT(*) FROM Zapato WHERE lote_prod_id =
@lote_act)
  WHERE lote_prod_id = @lote_act;

  COMMIT;
END TRY
BEGIN CATCH
  ROLLBACK;
  PRINT 'Error seccion G: ' + ERROR_MESSAGE();
END CATCH;
GO
```

H) Eliminar un diseño que ya tiene lote de producción

```
-- Si el diseño tiene producción asociada, no se elimina (se informa).
-- Si no tiene producción, se elimina en cascada.
BEGIN TRY
  BEGIN TRAN;

  DECLARE @dis_id INT = (SELECT diseno_id FROM Diseno
                         WHERE nombre_diseno = 'Modelo Actividad');

  IF EXISTS (
    SELECT 1 FROM Lote_Produccion lp
    JOIN   Diseno_Version dv ON lp.diseno_version_id = dv.diseno_version_id
    WHERE  dv.diseno_id = @dis_id
  )
    PRINT 'No se puede eliminar: el diseno tiene lotes de produccion asociados.';
  ELSE
  BEGIN
    DELETE FROM Diseno_Material WHERE diseno_version_id IN
      (SELECT diseno_version_id FROM Diseno_Version WHERE diseno_id = @dis_id);
    DELETE FROM Diseno_Version  WHERE diseno_id = @dis_id;
    DELETE FROM Diseno          WHERE diseno_id = @dis_id;
    PRINT 'Diseno eliminado correctamente.';
  END

  COMMIT;
END TRY
BEGIN CATCH
  ROLLBACK;
  PRINT 'Error seccion H: ' + ERROR_MESSAGE();
END CATCH;
GO
```

I) Insertar nuevo accesorio y diseño que lo use

```
BEGIN TRY
  BEGIN TRAN;
```

```
  -- Nuevo accesorio
  INSERT INTO Accesorio (nombre, descripcion)
  VALUES ('Cordones reflectivos', 'Cordones con banda reflectiva 120cm');
  DECLARE @acc2 INT = SCOPE_IDENTITY();

  -- Nuevo diseño
  INSERT INTO Diseno (nombre_diseno, descripcion, creado_por)
  VALUES ('Modelo Cordon', 'Zapato deportivo con cordones reflectivos', 1);
  DECLARE @dis2 INT = SCOPE_IDENTITY();

  -- Version 1 del nuevo diseño
  INSERT INTO Diseno_Version (diseno_id, version_num, creado_por, notas)
  VALUES (@dis2, 1, 1, 'Version inicial con cordones');
  DECLARE @dv_c INT = SCOPE_IDENTITY();

  -- Asociar accesorio
  INSERT INTO Diseno_Accesorio (diseno_version_id, accesorio_id, cantidad)
  VALUES (@dv_c, @acc2, 2);  -- 2 cordones por zapato

  COMMIT;
END TRY
BEGIN CATCH
  ROLLBACK;
  PRINT 'Error seccion I: ' + ERROR_MESSAGE();
END CATCH;
GO
```

Actividad 4: Consultas SQL

Consulta 1: Zapatos en los que se usó un molde determinado

Se busca por molde_id. Se puede cambiar el valor de @MoldeId por el identificador del molde que se quiera rastrear.

```
DECLARE @MoldeId INT = 1;  -- cambiar por el id del molde buscado

SELECT z.zapato_id,
       z.serial,
       z.talla,
       z.fecha_ensamblado,
       z.estado_zapato,
       lp.codigo_lote_prod
FROM   Zapato z
LEFT JOIN Lote_Produccion lp ON z.lote_prod_id = lp.lote_prod_id
WHERE  z.molde_usado_id = @MoldeId
ORDER BY z.fecha_ensamblado DESC;
```

Consulta 2: Lotes de material usados en la construcción de un zapato

Se identifica un zapato por su serial y se recorren los trozos que lo componen para obtener los lotes de material de origen.

```
DECLARE @Serial NVARCHAR(150) = 'ZAP-LP001-1';  -- serial del zapato

SELECT DISTINCT
       lm.lote_material_id,
       lm.codigo_lote,
       mat.nombre  AS material,
       lm.fecha_recepcion,
       lm.cantidad_inicial,
       lm.cantidad_disponible
FROM   Zapato z
JOIN   Trozo  t  ON t.zapato_id        = z.zapato_id
JOIN   Lote_Material lm ON t.lote_material_id = lm.lote_material_id
JOIN   Material mat     ON lm.material_id     = mat.material_id
WHERE  z.serial = @Serial
ORDER BY lm.lote_material_id;
```

Consulta 3: Cantidad de zapatos fabricados para un diseño

Muestra el total de zapatos de todas las versiones de un diseño, y también el desglose por versión.

```
-- Total por diseño
DECLARE @Nombre NVARCHAR(150) = 'Modelo Actividad';

SELECT d.diseno_id,
       d.nombre_diseno,
       COUNT(z.zapato_id) AS total_zapatos
FROM   Diseno d
LEFT JOIN Diseno_Version dv ON dv.diseno_id         = d.diseno_id
LEFT JOIN Zapato         z  ON z.diseno_version_id  = dv.diseno_version_id
WHERE  d.nombre_diseno = @Nombre
GROUP BY d.diseno_id, d.nombre_diseno;

-- Desglose por version
SELECT dv.version_num,
       dv.notas,
       COUNT(z.zapato_id) AS zapatos_en_version
FROM   Diseno d
JOIN   Diseno_Version dv ON dv.diseno_id        = d.diseno_id
LEFT JOIN Zapato      z  ON z.diseno_version_id = dv.diseno_version_id
WHERE  d.nombre_diseno = @Nombre
GROUP BY dv.diseno_version_id, dv.version_num, dv.notas
ORDER BY dv.version_num;
```